

Sección 3: Arquitectura en Capas con Criterio

- Por qué el entity no debe salir del service: DTOs de request y response
- Mapeo manual vs MapStruct
- Excepciones de dominio con ProblemDetail (RFC 7807)
- @RestControllerAdvice centralizado
- Bean Validation en DTOs: constraints estándar y mensajes custom
- Validaciones custom de negocio con @Constraint

25. Anatomía de un monolito bien hecho



→ Regla de dependencia:

- Controller conoce al Service — Service **NO** conoce al Controller
- Service conoce al Repository — Repository **NO** conoce al Service
- Las dependencias van siempre **hacia abajo**, nunca hacia arriba
- Si una capa inferior necesita comunicarse con una superior → eventos o interfaces
- Esta regla es la base de toda **arquitectura limpia**

→ ¿Qué va en cada capa?

Capa	Responsabilidad	Incluye	NO incluye
Controller	Recibir y responder HTTP	DTOs, validación de formato	Lógica de negocio
Service	Orquestar la lógica	Reglas de negocio, transacciones	Acceso a datos, HTTP
Repository	Persistencia	Queries, acceso a BD	Lógica de negocio, HTTP

26. DTOs: separando lo que exponés de lo que persistís

→ ¿Por qué la entity no debería llegar al cliente?

- El cliente ve la estructura interna de tu base de datos
- Si agregás un campo interno, aparece en la respuesta sin querer

- Si necesitás que la respuesta sea distinta a lo que guardás, no podés
- Estás acoplando tu contrato de API con tu modelo de persistencia
- Un cambio interno se filtra al exterior

→ DTO es un objeto que define exactamente qué datos entran y qué datos salen.

→ ¿Dónde vive el mapeo?

 Opción A — En el Service	 Opción B — En el Controller
• El Service recibe y devuelve DTOs • La entity nunca sale del service • El Controller es un pasamanos puro	• El Service devuelve la entity • El Controller mapea a DTO • El Service es más reutilizable

→ Opción A: DTO en el service:

```

El Service define el contrato con DTOs

public interface IAccountService {
    AccountResponse create(CreateAccountRequest request);
    AccountResponse findById(Long id);
}

// El Controller no mapea nada - solo pasa datos
@PostMapping
public ResponseEntity<AccountResponse> create(
    @RequestBody CreateAccountRequest request) {
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(accountService.create(request));
}

```

→ Opción B: DTO en el controller:

```

El Service devuelve entities, el Controller traduce

public interface IAccountService {
    Account create(Account account);
    Account findById(Long id);
}

// El Controller mapea entre DTOs y entities
@PostMapping
public ResponseEntity<AccountResponse> create(
    @RequestBody CreateAccountRequest request) {
    Account account = toEntity(request); // DTO → Entity
    Account saved = accountService.create(account);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(toResponse(saved)); // Entity → DTO
}

```

→ ¿Qué prioriza cada una?

	Opción A (DTO en Service)	Opción B (Entity en Service)
Prioriza	Encapsulación	Reutilización interna
Entity sale	No	Sí
Controller	Ultra liviano	Traduce entre DTO y entity
Contrato	Lo define el Service	Lo define el Controller
¿Está mal?	No	No

→ ¿Y en microservicios?

- Poca reutilización interna entre services
- El valor está en la API — el **contrato hacia afuera**
- El DTO pasa a ser parte del diseño del servicio
- Tiene sentido que el **Service sea el dueño** del contrato
- Pero la Opción B también funciona — no es un error

→ Se usará la opción B por ser monolito, por ahora.

27. DTOs de: request, response Y @Data

→ Account:

Project

atlas

new_projects\java\atlas

.idea

.mvn

src

main

java

com.atlas.bank.atlas

account

controller

AccountController

dto

AccountResponse

CreateAccountRequest

model

repository

service

transaction

controller

AccountResponse.java

```
1 package com.atlas.bank.atlas.account.dto;
2
3 import lombok.Data;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 @Data
9 public class AccountResponse {
10     private Long id;
11     private String accountNumber;
12     private String ownerName;
13     private String email;
14     private String type; // SAVING, CHECKING
15     private BigDecimal balance;
16     private String status; // ACTIVE, CLOSED, FROZEN
17     private LocalDateTime createdAt;
18 }
```

CreateAccountRequest.java

TransferRequest.java

TransactionResponse.java

TransactionController.java

1 package com.atlas.bank.atlas.account.dto;

2

3 import lombok.Data;

4

5 import java.math.BigDecimal;

6

7 // @Data => Genera getter, setter, toString, equalsAndHashCode, RequiredArgumentConstructor

8 // problema en las entity que tienen relaciones: Data usado en DTO no es problemático

9 @Data

10 public class CreateAccountRequest {

11 private String accountNumber;

12 private String ownerName;

13 private String email;

14 private String type;

15 private BigDecimal balance;

16 }

```
© CreateAccountRequest.java  © AccountController.java ×  © TransferRequest.java  © TransactionResponse.java  © Transe

22 public class AccountController {
26     @PostMapping no usages ydochoadev
27     public ResponseEntity<AccountResponse> create(@RequestBody CreateAccountRequest request) {
28         Account account = new Account();
29         account.setAccountNumber(request.getAccountNumber());
30         account.setOwnerName(request.getOwnerName());
31         account.setEmail(request.getEmail());
32         account.setType(request.getType());
33         account.setBalance(request.getBalance());
34
35         Account accountCreated = accountService.create(account);
36         return ResponseEntity.status(HttpStatus.CREATED).body(toResponse(accountCreated));
37     }
38
39     @GetMapping no usages ydochoadev
40     public ResponseEntity<List<AccountResponse>> findAll() {
41         List<AccountResponse> accountResponses = accountService.findAll() List<Account>
42             .stream() Stream<Account>
43             .map(this::toResponse) Stream<AccountResponse>
44             .toList();
45
46         return ResponseEntity.ok(accountResponses);
47     }
48
49     @GetMapping("/{id}") no usages ydochoadev
50     public ResponseEntity<AccountResponse> findById(@PathVariable Long id) {
51         return ResponseEntity.ok(toResponse(accountService.findById(id)));
52     }
53
54     @private AccountResponse toResponse(Account account) { 3 usages ydochoadev
55         AccountResponse response = new AccountResponse();
56         response.setId(account.getId());
57         response.setAccountNumber(account.getAccountNumber());
58         response.setOwnerName(account.getOwnerName());
59         response.setEmail(account.getEmail());
60         response.setType(account.getType());
61         response.setBalance(account.getBalance());
62         response.setStatus(account.getStatus());
63         response.setCreatedAt(account.getCreatedAt());
64     }
```

➔Transaction o transfer:

```
Project v
└─ atlas D:\new_projects\java\atlas
   └─ .idea
      └─ .mvn
         └─ src
            └─ main
               └─ java
                  └─ com.atlas.bank.atlas
                     └─ account
                        └─ transaction
                           └─ controller
                              © TransactionController
                           └─ dto
                              © TransactionResponse
                              © TransferRequest
                           └─ model
                              © Transaction

© TransferRequest.java ×  © TransactionResponse.java  © TransactionController.java

1 package com.atlas.bank.atlas.transaction.dto;
2
3 import lombok.Data;
4
5 import java.math.BigDecimal;
6
7 @Data 2 usages
8 public class TransferRequest {
9     private String type; // DEPOSIT, WITHDRAWAL, TRANSFER
10    private Long fromAccountId;
11    private Long toAccountId;
12    private BigDecimal amount;
13 }
14
```

The screenshot shows an IDE with a project structure on the left and a Java class definition on the right. The project structure is for a project named 'atlas' located at 'D:\new_projects\java\atlas'. It includes folders for '.idea', '.mvn', 'src', 'main', 'java', 'com.atlas.bank.atlas', 'account', 'transaction', 'controller', 'dto', 'model', 'repository', 'service', and 'AtlasApplication'. The 'TransactionResponse' class is highlighted in the 'dto' folder. The class definition on the right is as follows:

```
1 package com.atlas.bank.atlas.transaction.dto;
2
3 import lombok.Data;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 @Data 7 usages
9 public class TransactionResponse {
10     private Long id;
11     private String type;
12     private Long sourceAccountId;
13     private Long targetAccountId;
14     private BigDecimal amount;
15     private BigDecimal fee;
16     private String status;
17     private LocalDateTime createdAt;
18 }
19
```

The screenshot shows an IDE with a Java class definition for 'TransactionController'. The class is defined as follows:

```
22 public class TransactionController {
23
24     private final ITransactionQueryService transactionQueryService;
25
26     @PostMapping("/transfer") no usages new *
27     @RequestBody
28     public ResponseEntity<TransactionResponse> transfer(@RequestBody TransferRequest request) {
29         Transaction transaction = transferService.execute(
30             request.getFromAccountId(),
31             request.getToAccountId(),
32             request.getAmount()
33         );
34
35         return ResponseEntity.ok(toResponse(transaction));
36     }
37
38     @GetMapping("/{id}/transactions") no usages new *
39     public ResponseEntity<List<TransactionResponse>> getTransactions(@PathVariable Long id) {
40         List<TransactionResponse> transactionResponseList = transactionQueryService.getByAccountId(id).list();
41         .stream() Stream<Transaction>
42         .map(this::toResponse) Stream<TransactionResponse>
43         .toList();
44
45         return ResponseEntity.ok(transactionResponseList);
46     }
47
48     TransactionResponse toResponse(Transaction t) { 2 usages new *
49         TransactionResponse response = new TransactionResponse();
50         response.setId(t.getId());
51         response.setType(t.getType());
52         response.setSourceAccountId(t.getSourceAccountId());
53         response.setTargetAccountId(t.getTargetAccountId());
54         response.setAmount(t.getAmount());
55         response.setFee(t.getFee());
56         response.setStatus(t.getStatus());
57         response.setCreatedAt(t.getCreatedAt());
58
59         return response;
60     }
61 }
62
```

28. Mapeo de DTOs: MapStruct

➔ Agregando dependencia de mapstruct.

```
m pom.xml (atlas) x
2   <project xmlns="http://maven.apache.org/POM/4.0.0" xm
23   <scm>
27       <url/>
28   </scm>
29   <properties>
30       <java.version>21</java.version>
31       <!-- Agregando propiedad mapstruct -->
32       <mapstruct.version>1.6.3</mapstruct.version>
33   </properties>
34   <dependencies> Add Starters...
35   <dependency>
```

```
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</ar
</dependency>
<!-- Agregando dependencia mapstruct -->
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>${mapstruct.version}</version>
</dependency>
```

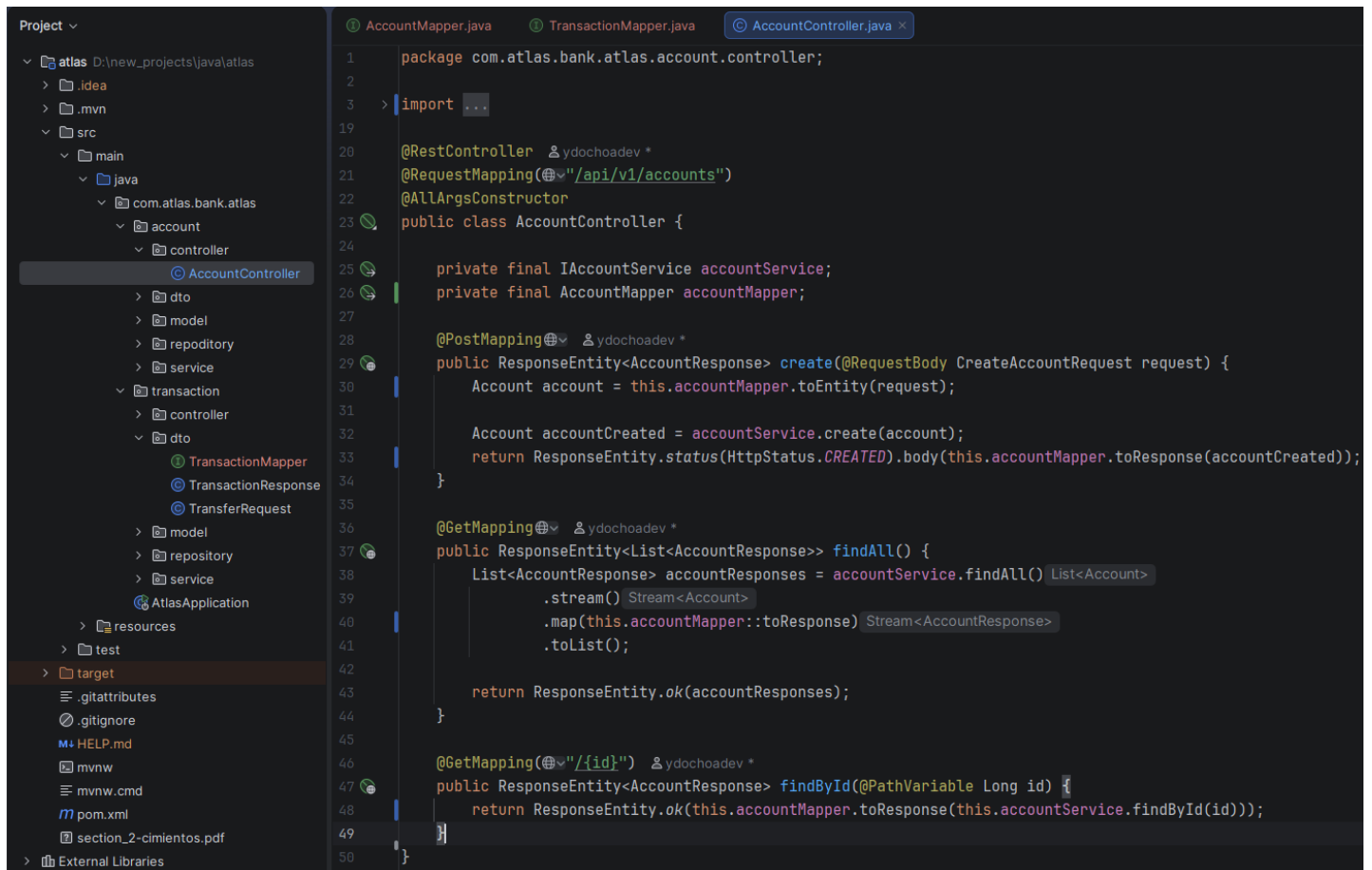
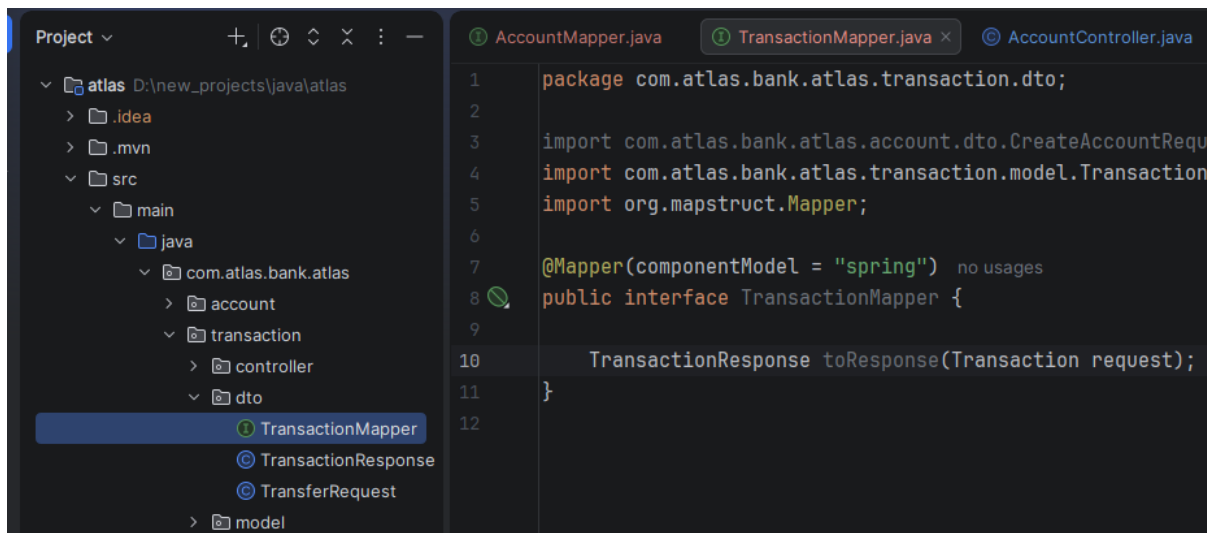
```
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <executions>
        <execution>
            <id>default-compile</id>
            <phase>compile</phase>
            <goals>
                <goal>compile</goal>
            </goals>
            <configuration>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                    <!-- Agregando path mapstruct -->
                    <path>
                        <groupId>org.mapstruct</groupId>
                        <artifactId>mapstruct-processor</artifactId>
                        <version>${mapstruct.version}</version>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </execution>
    </executions>
```

➔ Agregando interface mapper en Account y Transaction y modificando controller de Account:

The screenshot shows an IDE with the project structure on the left and the code for AccountMapper.java on the right. The project structure includes a package com.atlas.bank.atlas with sub-packages account, controller, and dto. The AccountMapper.java file is located in the dto package. The code defines the AccountMapper interface with two methods: toEntity and toResponse. The toEntity method takes a CreateAccountRequest and returns an Account. The toResponse method takes an Account and returns an AccountResponse. The interface is annotated with @Mapper(componentModel = "spring").

```
Project v
└─ atlas D:\new_projects\java\atlas
   └─ .idea
   └─ .mvn
   └─ src
      └─ main
         └─ java
            └─ com.atlas.bank.atlas
               └─ account
                  └─ controller
                     └─ AccountController
                  └─ dto
                     └─ AccountMapper
                     └─ AccountResponse
                     └─ CreateAccountRequest
                  └─ model
                  └─ repository
                  └─ service
```

```
AccountMapper.java x
1 package com.atlas.bank.atlas.account.dto;
2
3 import com.atlas.bank.atlas.account.model.Account;
4 import org.mapstruct.Mapper;
5 import org.mapstruct.Mapping;
6
7 @Mapper(componentModel = "spring") 2 usages
8 public interface AccountMapper {
9
10     @Mapping(target = "id", ignore = true) 1 usage
11     @Mapping(target = "status", ignore = true)
12     @Mapping(target = "createdAt", ignore = true)
13     Account toEntity(CreateAccountRequest request);
14
15     AccountResponse toResponse(Account account); 3 usages
16 }
17
```



➔ Modificando controlador de Transaction:

```
1 package com.atlas.bank.atlas.transaction.controller;
2
3 > import ...
4
19
20 @RestController @ydochoadev *
21 @RequestMapping(@"/api/v1/transactions")
22 @RequiredArgsConstructor
23 public class TransactionController {
24
25     private final ITransferService transferService;
26     private final ITransactionQueryService transactionQueryService;
27     private final TransactionMapper transactionMapper;
28
29     @PostMapping(@"/transfer") @ydochoadev *
30     public ResponseEntity<TransactionResponse> transfer(@RequestBody TransferRequest request) {
31         Transaction transaction = transferService.execute(
32             request.getFromAccountId(),
33             request.getToAccountId(),
34             request.getAmount()
35         );
36
37         return ResponseEntity.ok(this.transactionMapper.toResponse(transaction));
38     }
39
40     @GetMapping(@"/{id}/transactions") @ydochoadev *
41     public ResponseEntity<List<TransactionResponse>> getTransactions(@PathVariable Long id) {
42         List<TransactionResponse> transactionResponseList = transactionQueryService.getByAccountId(id).L
43             .stream() Stream<Transaction>
44             .map(this.transactionMapper::toResponse) Stream<TransactionResponse>
45             .toList();
46
47         return ResponseEntity.ok(transactionResponseList);
48     }
49
50 }
```

30. ProblemDetail: errores con estándar RFC 7807

→ RuntimeException vs ProblemDetail

Hoy — RuntimeException	Con ProblemDetail — RFC 7807
<pre>HTTP 500 Internal Server Error { "error": "Internal Server Error", "message": "Fondos insuficientes" } // ¿Error del usuario? // ¿Error del servidor? // No hay forma de saberlo</pre>	<pre>HTTP 422 Unprocessable Entity { "type": "about:blank", "title": "Fondos insuficientes", "status": 422, "detail": "La cuenta 1 no tiene fondos suficientes para transferir 5000.00" }</pre>

→ ProblemDetail es un estándar. Creando excepciones en cuenta y transacciones:

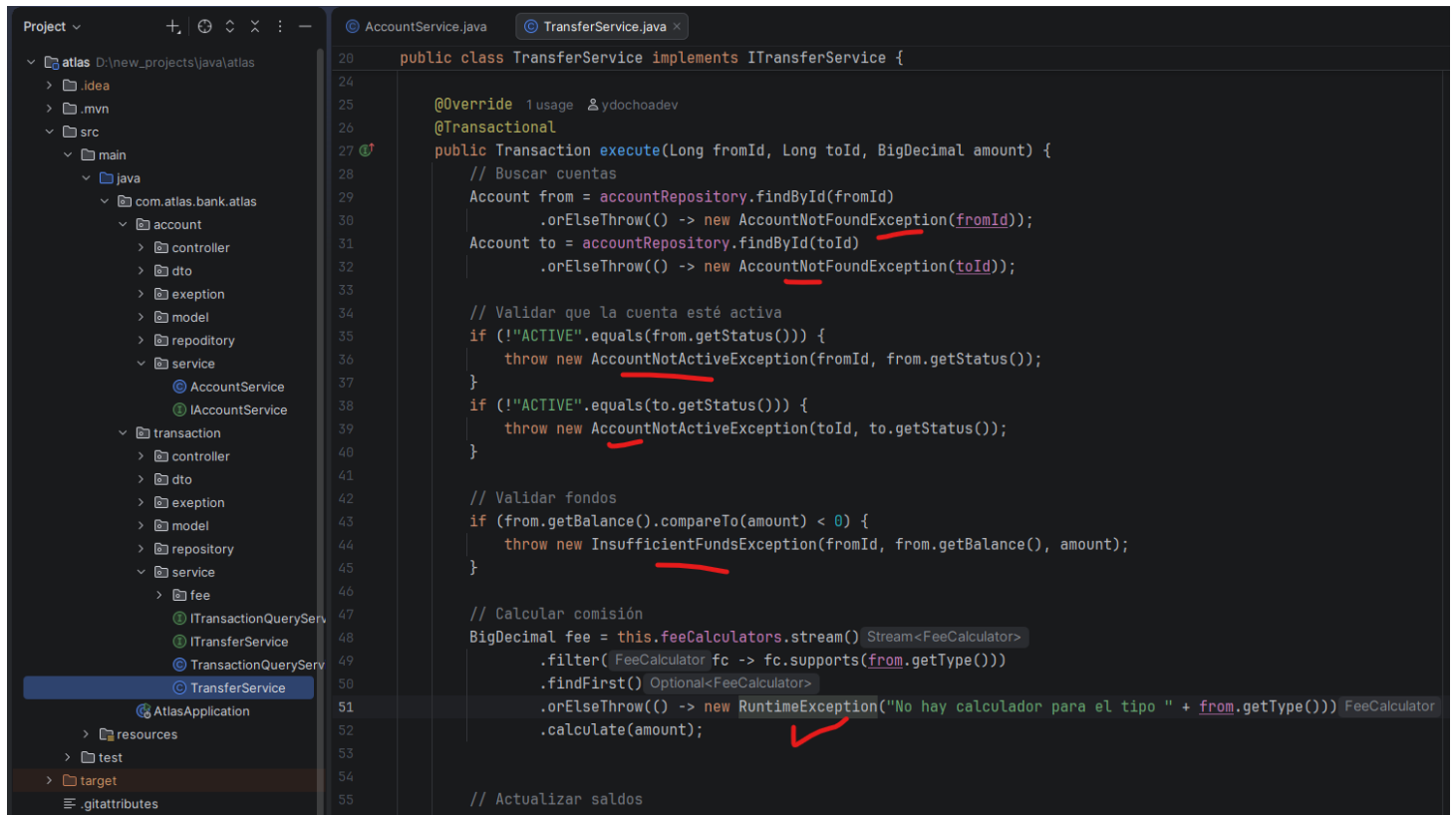
```
1 package com.atlas.bank.atlas.account.expection;
2
3
4 > public class AccountNotFoundException extends RuntimeException { no usages
5     public AccountNotFoundException(Long id) { super("No se encontró la cuenta con id: " + id); }
6
7 }
8
```

```
1 package com.atlas.bank.atlas.transaction.expection;
2
3 import java.math.BigDecimal;
4
5 public class InsufficientFundsException extends RuntimeException { no usages
6     public InsufficientFundsException(Long accountId, BigDecimal balance, BigDecimal amount) { no usages
7         super("La cuenta " + accountId + " tiene un saldo de " + balance + " y se intentó transferir " + amount);
8     }
9 }
10
```

```
1 package com.atlas.bank.atlas.transaction.expection;
2
3 public class AccountNotActiveException extends RuntimeException { no usages
4     public AccountNotActiveException(Long accountId, String status) { no usages
5         super("La cuenta " + accountId + " no está activa. Estado actual: " + status);
6     }
7 }
8
```

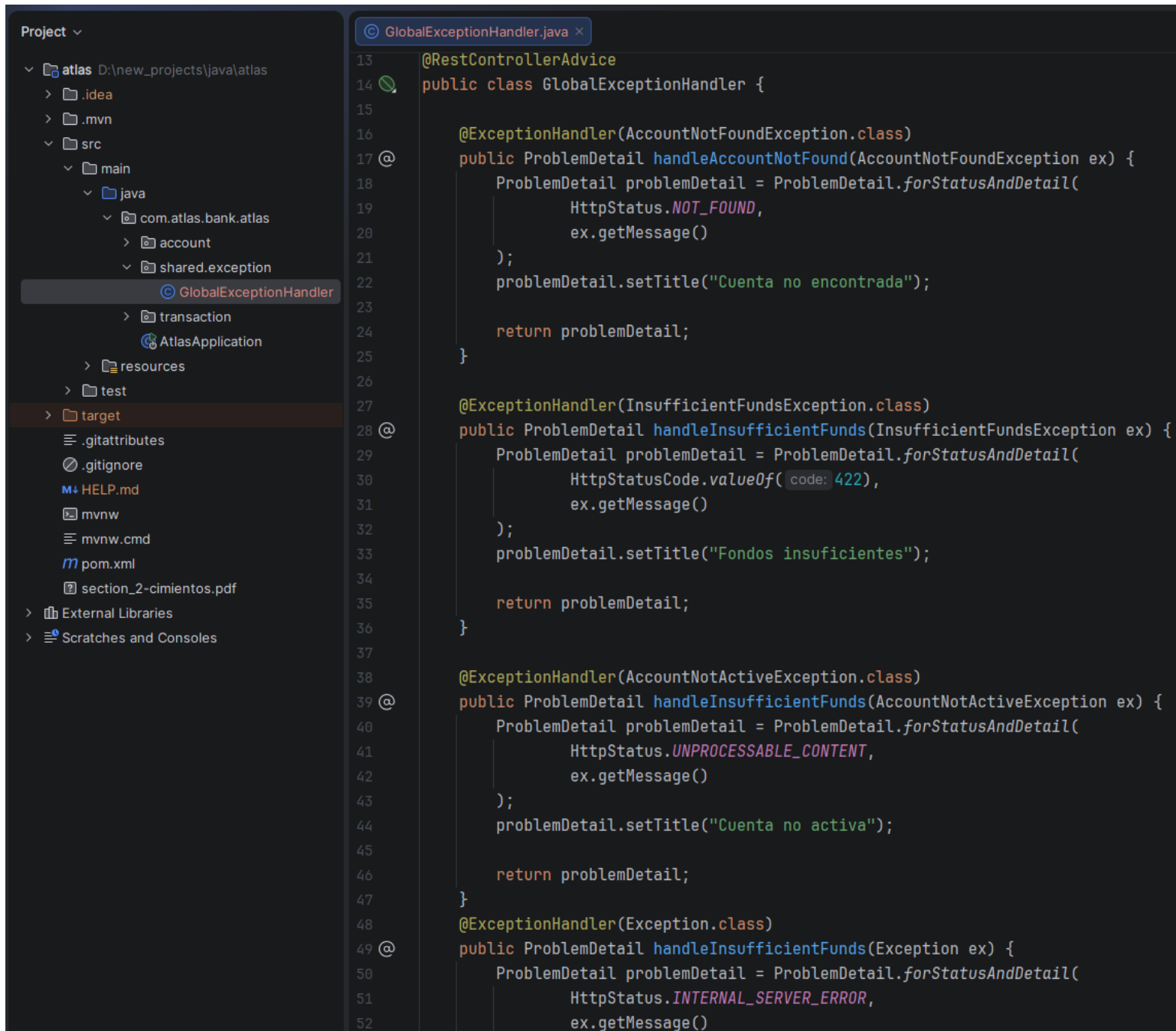
➔Corrigiendo los servicios:

```
1 package com.atlas.bank.atlas.account.service;
2
3 > import ...
4
10
11 @Service & ydochoadev
12 @RequiredArgsConstructor
13 public class AccountService implements IAccountService {
14
15     private final AccountRepository accountRepository;
16
17     @Override 1 usage & ydochoadev
18     public Account create(Account account) { return accountRepository.save(account); }
19
20
21
22     @Override 1 usage & ydochoadev
23     public List<Account> findAll() { return accountRepository.findAll(); }
24
25
26
27     @Override 1 usage & ydochoadev
28     public Account findById(Long id) {
29         return accountRepository.findById(id).orElseThrow(() -> new AccountNotFoundException(id));
30     }
31
32 }
```



31. @RestControllerAdvice centralizado

➔ Se tienen las excepciones, pero spring no sabe qué hacer con estas. Las trata como cualquier excepción no gestionada. Se necesita un lugar centralizado que atrape cada una de las excepciones y la convierta a ProblemDetail con el status code correcto.



**** NO SE DEBEN MOSTRAR ERRORES DEL SISTEMA**

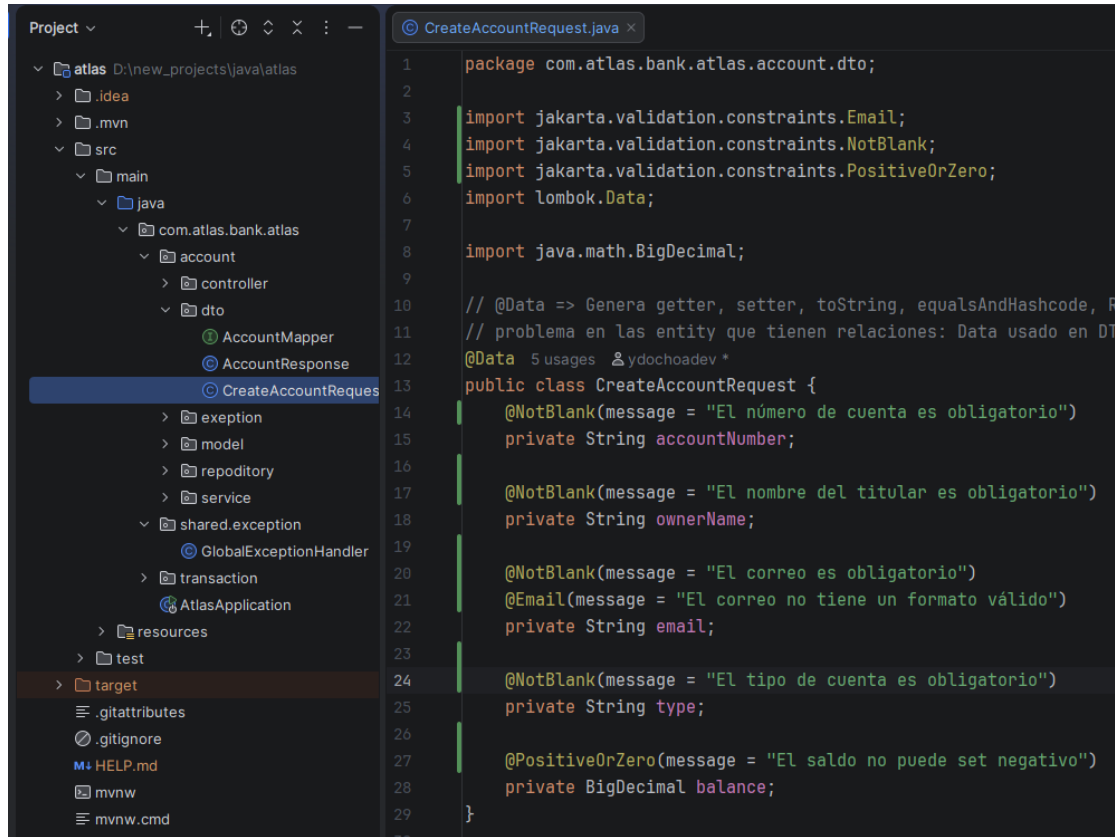
```
@ExceptionHandler(Exception.class)
public ProblemDetail handleInsufficientFunds(Exception ex) {
    ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(
        HttpStatus.INTERNAL_SERVER_ERROR,
        detail: "Error interno del servidor"
    );
    problemDetail.setTitle("Error interno");

    return problemDetail;
}
```

32. Bean validation en DTOs

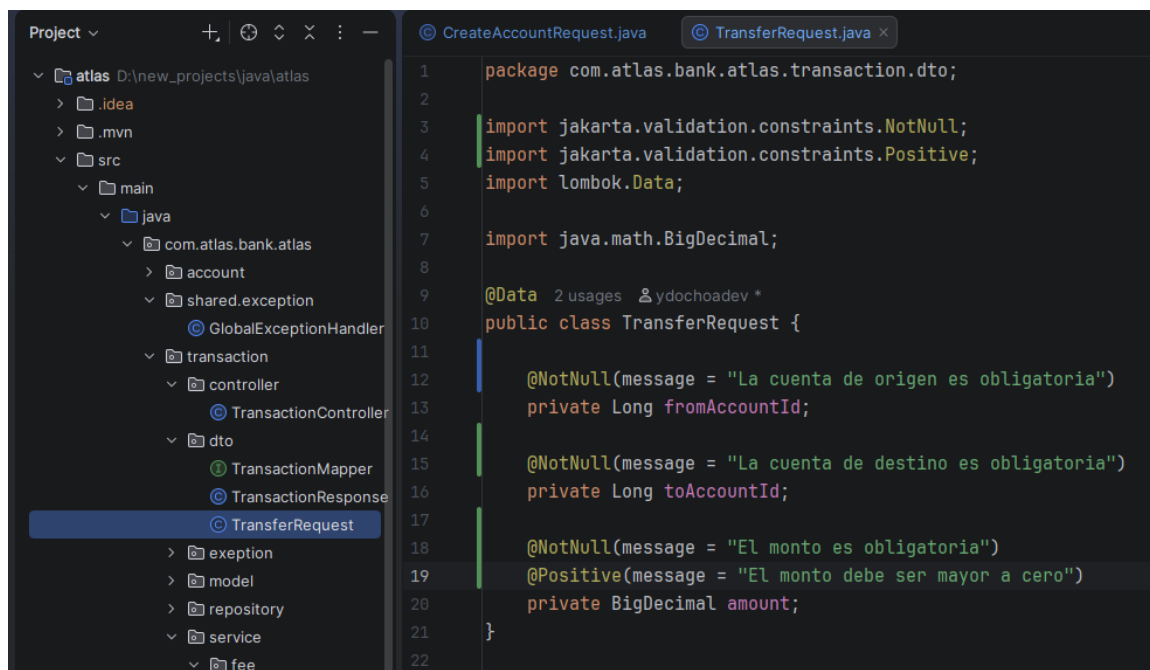
➔Primera línea de defensa.

```
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
</dependency>
```



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a package `com.atlas.bank.atlas` with sub-packages `account`, `controller`, `dto`, `expection`, `model`, `repository`, `service`, `shared.exception`, `transaction`, and `resources`. The `dto` package contains `AccountMapper`, `AccountResponse`, `CreateAccountRequest`, `GlobalExceptionHandler`, and `AtlasApplication`. The `transaction` package contains `TransactionController`, `TransactionMapper`, `TransactionResponse`, and `TransferRequest`. The `dto` package also contains `expection`, `model`, `repository`, `service`, `shared.exception`, `transaction`, and `resources`. The `transaction` package contains `TransactionController`, `TransactionMapper`, `TransactionResponse`, and `TransferRequest`.

```
1 package com.atlas.bank.atlas.account.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.PositiveOrZero;
6 import lombok.Data;
7
8 import java.math.BigDecimal;
9
10 // @Data => Genera getter, setter, toString, equalsAndHashcode, R
11 // problema en las entity que tienen relaciones: Data usado en DT
12 @Data 5 usages ydochoadev *
13 public class CreateAccountRequest {
14     @NotBlank(message = "El número de cuenta es obligatorio")
15     private String accountNumber;
16
17     @NotBlank(message = "El nombre del titular es obligatorio")
18     private String ownerName;
19
20     @NotBlank(message = "El correo es obligatorio")
21     @Email(message = "El correo no tiene un formato válido")
22     private String email;
23
24     @NotBlank(message = "El tipo de cuenta es obligatorio")
25     private String type;
26
27     @PositiveOrZero(message = "El saldo no puede set negativo")
28     private BigDecimal balance;
29 }
```



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a package `com.atlas.bank.atlas` with sub-packages `account`, `controller`, `dto`, `expection`, `model`, `repository`, `service`, `shared.exception`, `transaction`, and `resources`. The `dto` package contains `AccountMapper`, `AccountResponse`, `CreateAccountRequest`, `GlobalExceptionHandler`, and `AtlasApplication`. The `transaction` package contains `TransactionController`, `TransactionMapper`, `TransactionResponse`, and `TransferRequest`. The `dto` package also contains `expection`, `model`, `repository`, `service`, `shared.exception`, `transaction`, and `resources`. The `transaction` package contains `TransactionController`, `TransactionMapper`, `TransactionResponse`, and `TransferRequest`.

```
1 package com.atlas.bank.atlas.transaction.dto;
2
3 import jakarta.validation.constraints.NotNull;
4 import jakarta.validation.constraints.Positive;
5 import lombok.Data;
6
7 import java.math.BigDecimal;
8
9 @Data 2 usages ydochoadev *
10 public class TransferRequest {
11
12     @NotNull(message = "La cuenta de origen es obligatoria")
13     private Long fromAccountId;
14
15     @NotNull(message = "La cuenta de destino es obligatoria")
16     private Long toAccountId;
17
18     @NotNull(message = "El monto es obligatoria")
19     @Positive(message = "El monto debe ser mayor a cero")
20     private BigDecimal amount;
21 }
22
```

→ En el controller

```
@RestController @ydochoadev *
@RequestMapping("/api/v1/accounts")
@AllArgsConstructor
public class AccountController {

    private final IAccountService accountService;
    private final AccountMapper accountMapper;

    @PostMapping @ydochoadev *
    public ResponseEntity<AccountResponse> create(@Valid @RequestBody CreateAccountRequest request) {
        Account account = this.accountMapper.toEntity(request);

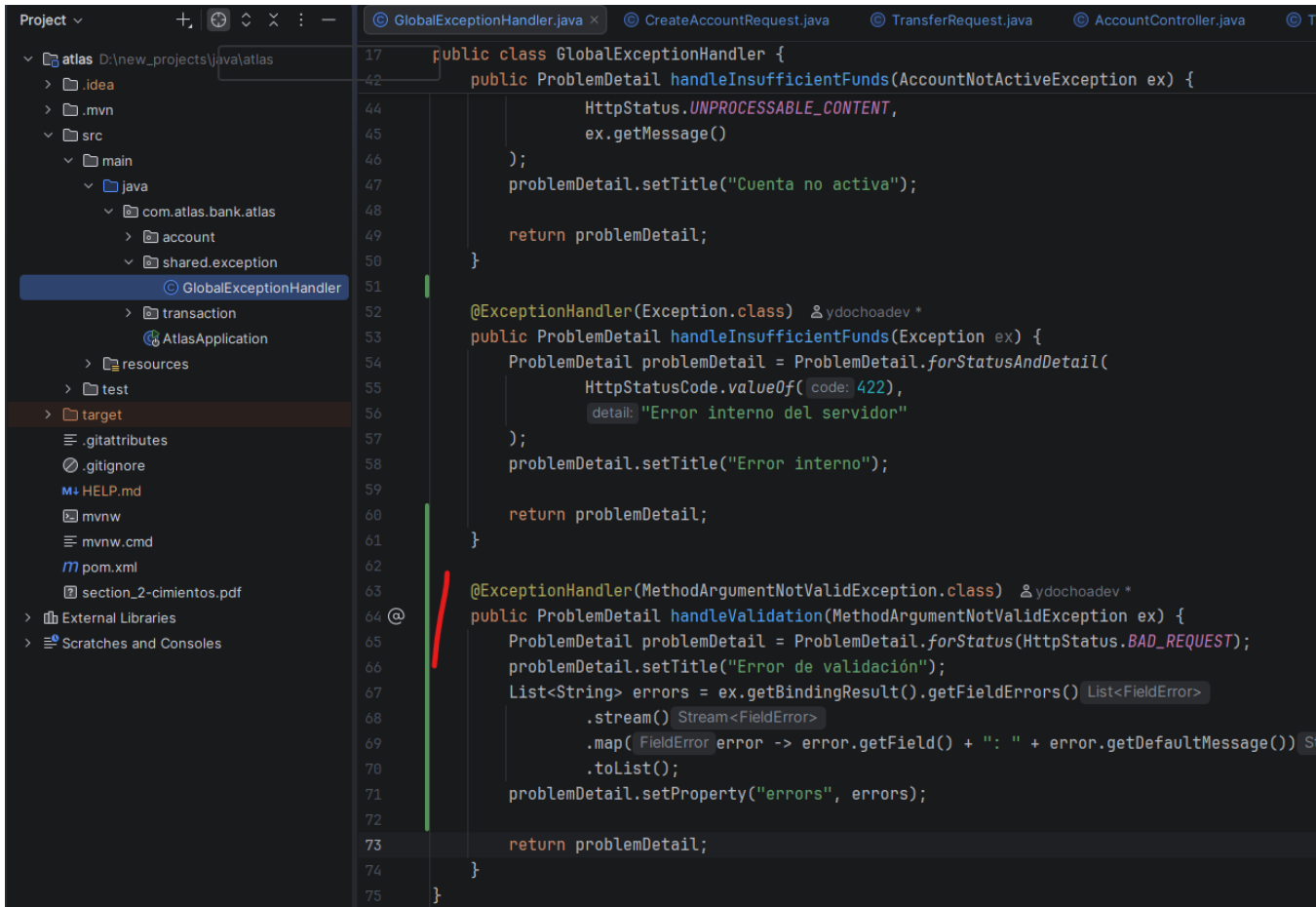
        Account accountCreated = accountService.create(account);
        return ResponseEntity.status(HttpStatus.CREATED).body(this.accountMapper.toResponse(accountCreated));
    }
}
```

```
@RestController @ydochoadev *
@RequestMapping("/api/v1/transactions")
@RequiredArgsConstructor
public class TransactionController {

    private final ITransferService transferService;
    private final ITransactionQueryService transactionQueryService;
    private final TransactionMapper transactionMapper;

    @PostMapping("/transfer") @ydochoadev *
    public ResponseEntity<TransactionResponse> transfer(@Valid @RequestBody TransferRequest request) {
        Transaction transaction = transferService.execute(
            request.getFromAccountId(),
            request.getToAccountId(),
            request.getAmount()
        );
    }
}
```

→ Excepción global



```
Project
├── atlas
│   ├── .idea
│   ├── .mvn
│   ├── src
│   │   ├── main
│   │   │   ├── java
│   │   │   │   ├── com.atlas.bank.atlas
│   │   │   │   │   ├── account
│   │   │   │   │   ├── shared.exception
│   │   │   │   │   │   ├── GlobalExceptionHandler
│   │   │   │   │   │   ├── transaction
│   │   │   │   │   │   └── AtlasApplication
│   │   │   │   ├── resources
│   │   │   └── test
│   │   └── target
│   ├── .gitattributes
│   ├── .gitignore
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   └── section_2-cimientos.pdf
├── External Libraries
└── Scratches and Consoles

GlobalExceptionHandler.java
17 public class GlobalExceptionHandler {
42     public ProblemDetail handleInsufficientFunds(AccountNotActiveException ex) {
44         HttpStatus.UNPROCESSABLE_CONTENT,
45         ex.getMessage()
46     );
47     problemDetail.setTitle("Cuenta no activa");
48
49     return problemDetail;
50 }
51
52 @ExceptionHandler(Exception.class) @ydochoadev *
53 public ProblemDetail handleInsufficientFunds(Exception ex) {
54     ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(
55         HttpStatus.valueOf( code: 422),
56         detail: "Error interno del servidor"
57     );
58     problemDetail.setTitle("Error interno");
59
60     return problemDetail;
61 }
62
63 @ExceptionHandler(MethodArgumentNotValidException.class) @ydochoadev *
64 @
65 public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
66     ProblemDetail problemDetail = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
67     problemDetail.setTitle("Error de validación");
68     List<String> errors = ex.getBindingResult().getFieldErrors().stream()
69         .map( FieldError error -> error.getField() + ": " + error.getDefaultMessage())
70         .toList();
71     problemDetail.setProperty("errors", errors);
72
73     return problemDetail;
74 }
75 }
```

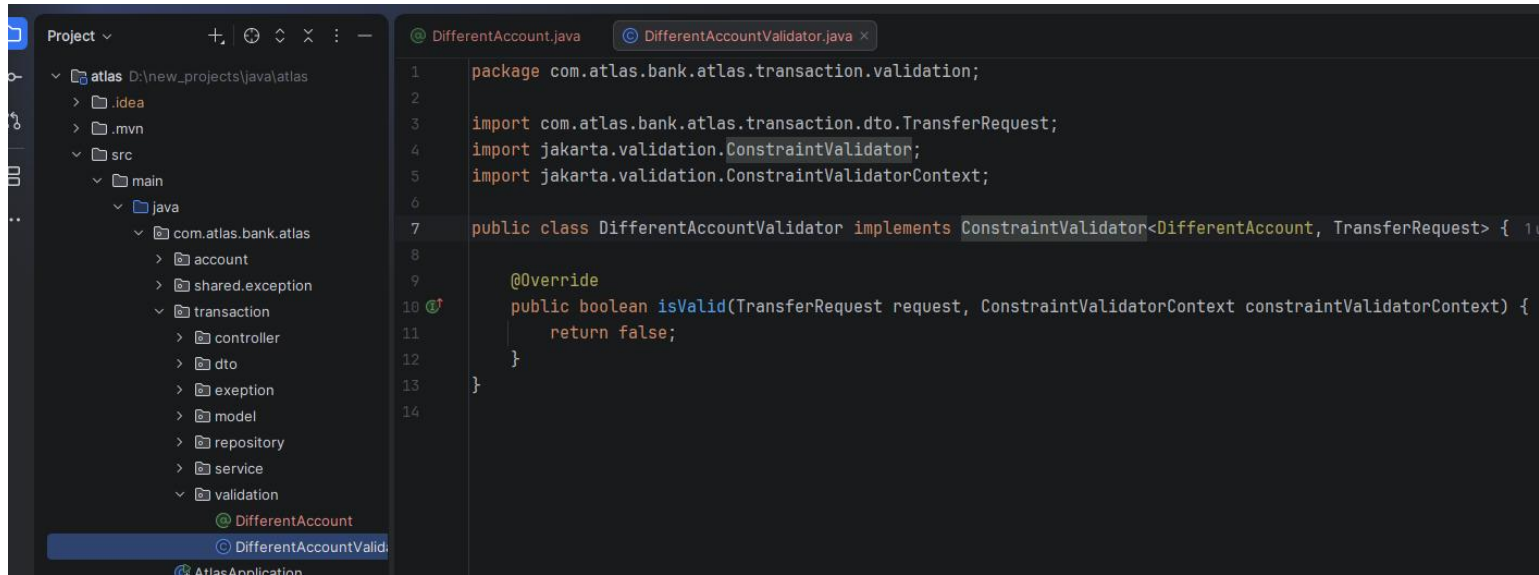
33. Validaciones custom de petición - parte 1

→ La cuenta de origen debe ser diferente a la cuenta destino.

→ Crear una clase de tipo anotación, anotación de clase (que irá arriba de la clase - @Target(ElementType.TYPE))

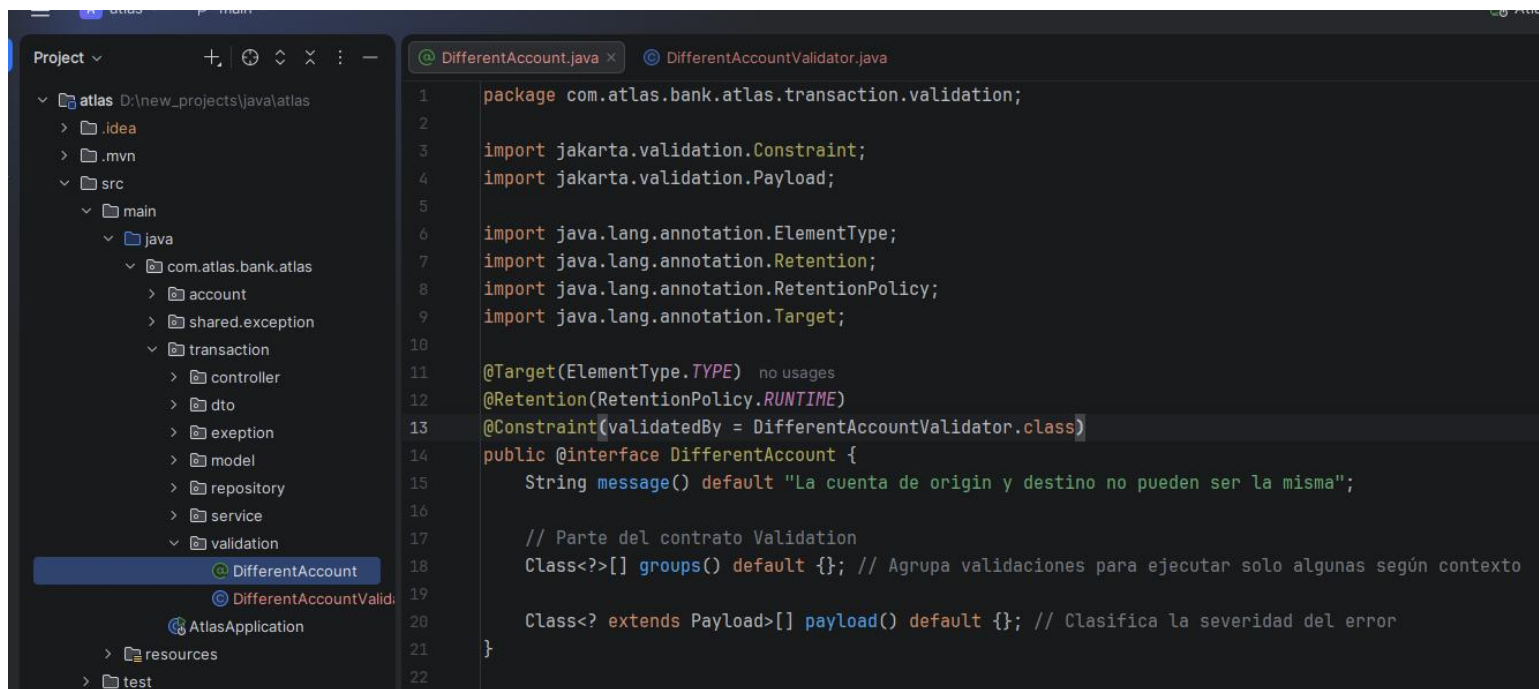
@Retention(RetentionPolicy.RUNTIME) => Hasta cuándo vive la anotación, en tiempo de ejecución

@Constraint(validatedBy = {DifferentAccountValidator.class}) => Ejecuta la lógica de validación



The screenshot shows the IntelliJ IDEA IDE with the 'Project' view on the left and the 'DifferentAccountValidator.java' file open in the editor. The project structure includes 'com.atlas.bank.atlas' with sub-packages like 'account', 'shared.exception', 'transaction', 'controller', 'dto', 'expection', 'model', 'repository', 'service', and 'validation'. The 'validation' package contains 'DifferentAccount' and 'DifferentAccountValidator'. The code in 'DifferentAccountValidator.java' is as follows:

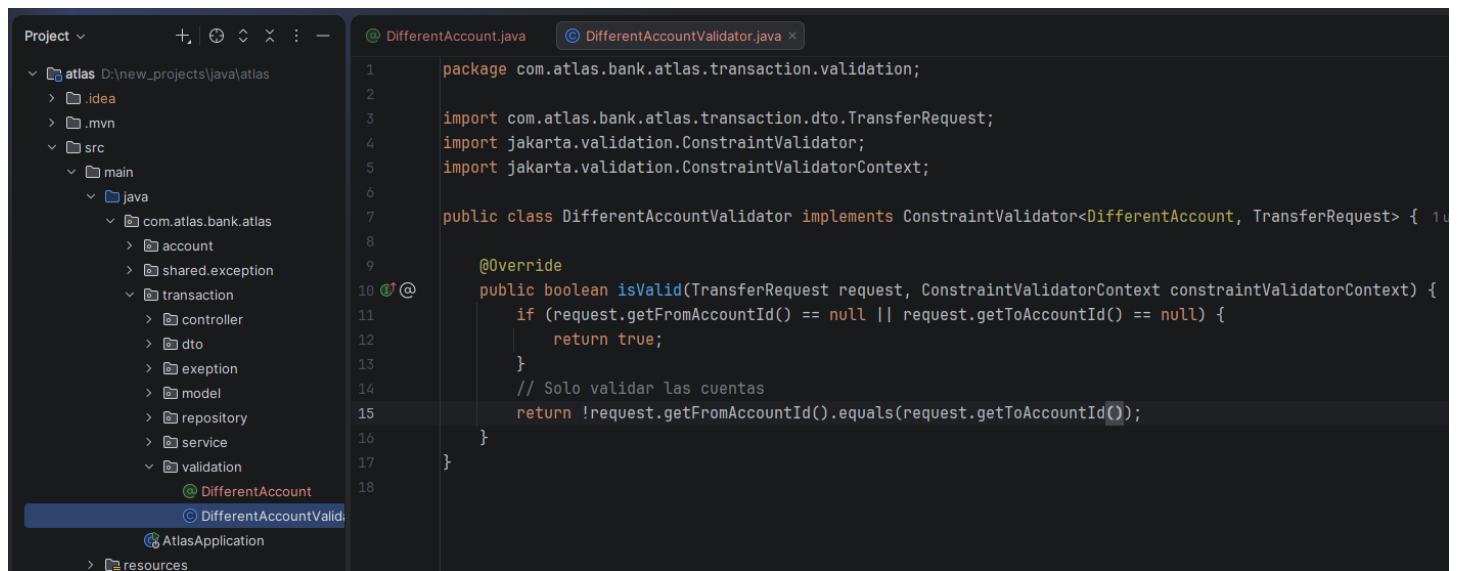
```
1 package com.atlas.bank.atlas.transaction.validation;
2
3 import com.atlas.bank.atlas.transaction.dto.TransferRequest;
4 import jakarta.validation.ConstraintValidator;
5 import jakarta.validation.ConstraintValidatorContext;
6
7 public class DifferentAccountValidator implements ConstraintValidator<DifferentAccount, TransferRequest> {
8
9     @Override
10    public boolean isValid(TransferRequest request, ConstraintValidatorContext constraintValidatorContext) {
11        return false;
12    }
13 }
14
```



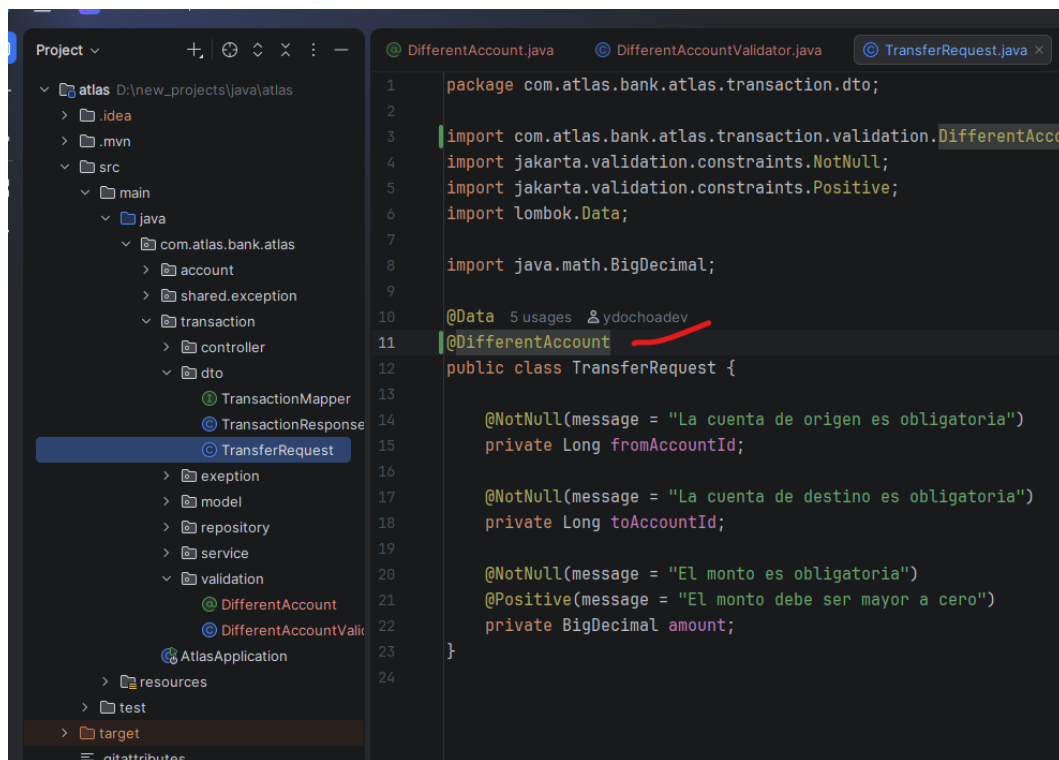
The screenshot shows the IntelliJ IDEA IDE with the 'Project' view on the left and the 'DifferentAccount.java' file open in the editor. The project structure is the same as the previous screenshot. The code in 'DifferentAccount.java' is as follows:

```
1 package com.atlas.bank.atlas.transaction.validation;
2
3 import jakarta.validation.Constraint;
4 import jakarta.validation.Payload;
5
6 import java.lang.annotation.ElementType;
7 import java.lang.annotation.Retention;
8 import java.lang.annotation.RetentionPolicy;
9 import java.lang.annotation.Target;
10
11 @Target(ElementType.TYPE) no usages
12 @Retention(RetentionPolicy.RUNTIME)
13 @Constraint(validatedBy = DifferentAccountValidator.class)
14 public @interface DifferentAccount {
15     String message() default "La cuenta de origen y destino no pueden ser la misma";
16
17     // Parte del contrato Validation
18     Class<?>[] groups() default {}; // Agrupa validaciones para ejecutar solo algunas según contexto
19
20     Class<? extends Payload>[] payload() default {}; // Clasifica la severidad del error
21 }
22
```

34. Validaciones custom de petición - parte 2

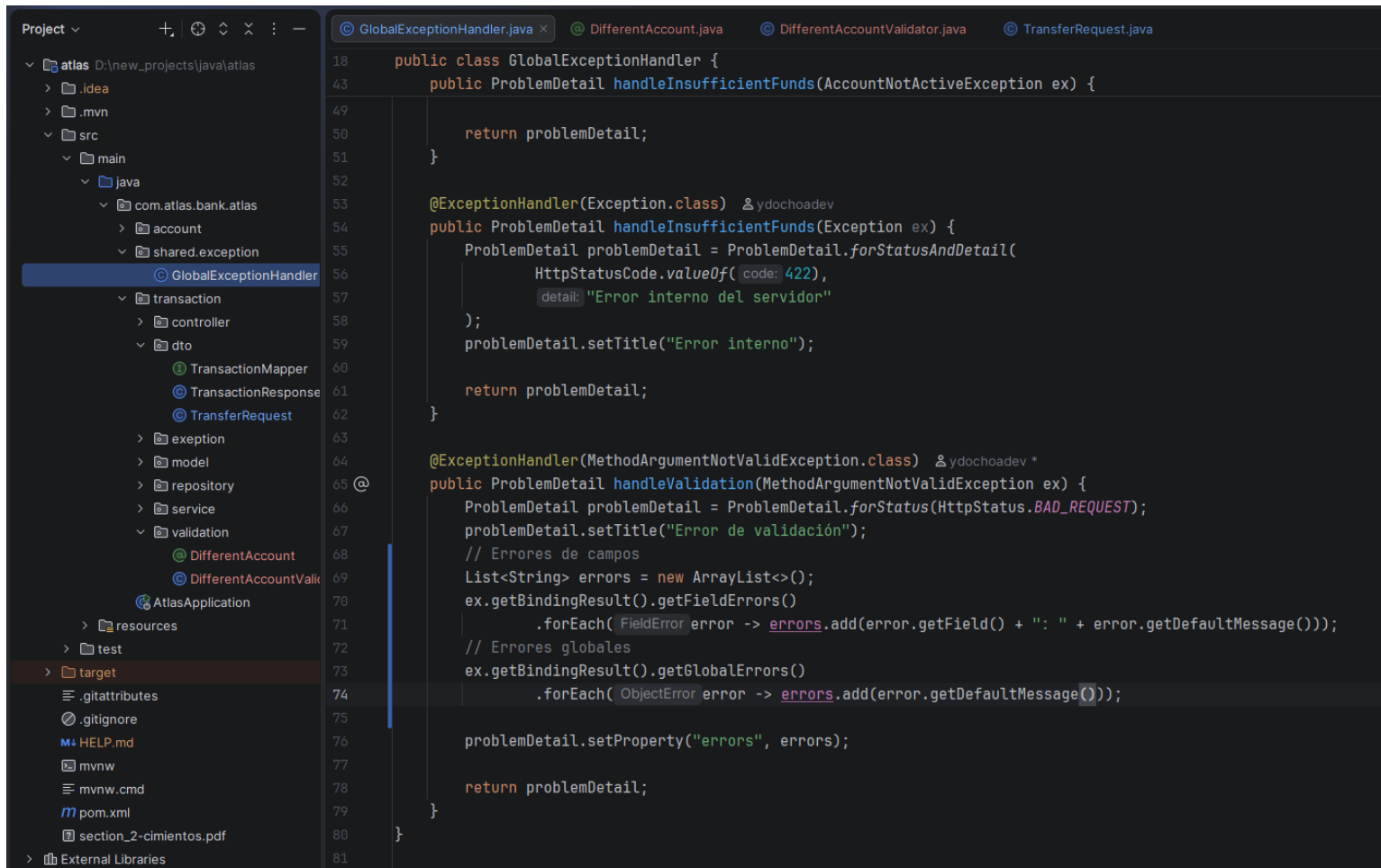


→ Colocar la anotación:



➔ Pero en el manejador global de excepciones, handleValidation, valida los campos, no a nivel de clase.

Refactorizando:



```
18 public class GlobalExceptionHandler {
43     public ProblemDetail handleInsufficientFunds(AccountNotActiveException ex) {
49
50         return problemDetail;
51     }
52
53     @ExceptionHandler(Exception.class) @ydochoadev
54     public ProblemDetail handleInsufficientFunds(Exception ex) {
55         ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(
56             HttpStatusCode.valueOf( code: 422),
57             detail: "Error interno del servidor"
58         );
59         problemDetail.setTitle("Error interno");
60
61         return problemDetail;
62     }
63
64     @ExceptionHandler(MethodArgumentNotValidException.class) @ydochoadev *
65     public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
66         ProblemDetail problemDetail = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
67         problemDetail.setTitle("Error de validación");
68         // Errores de campos
69         List<String> errors = new ArrayList<>();
70         ex.getBindingResult().getFieldErrors()
71             .forEach( FieldError error -> errors.add(error.getField() + ": " + error.getDefaultMessage()));
72         // Errores globales
73         ex.getBindingResult().getGlobalErrors()
74             .forEach( ObjectError error -> errors.add(error.getDefaultMessage()));
75
76         problemDetail.setProperty("errors", errors);
77
78         return problemDetail;
79     }
80 }
81
```